

SafeCross

Pedestrian Safety Technology

AI-Based Smart Crosswalk System · Full-Stack Project

TEAM MEMBERS

Netanel Kadoshi
Or Smolarchik
Rachel Brodsky
Yosef Yosef
Liel Babayn

INSTITUTION

HIT — Holon Institute of
Technology

DATE

30/09/2026

PROJECT PAGE

[HIT Projects Center →](#)

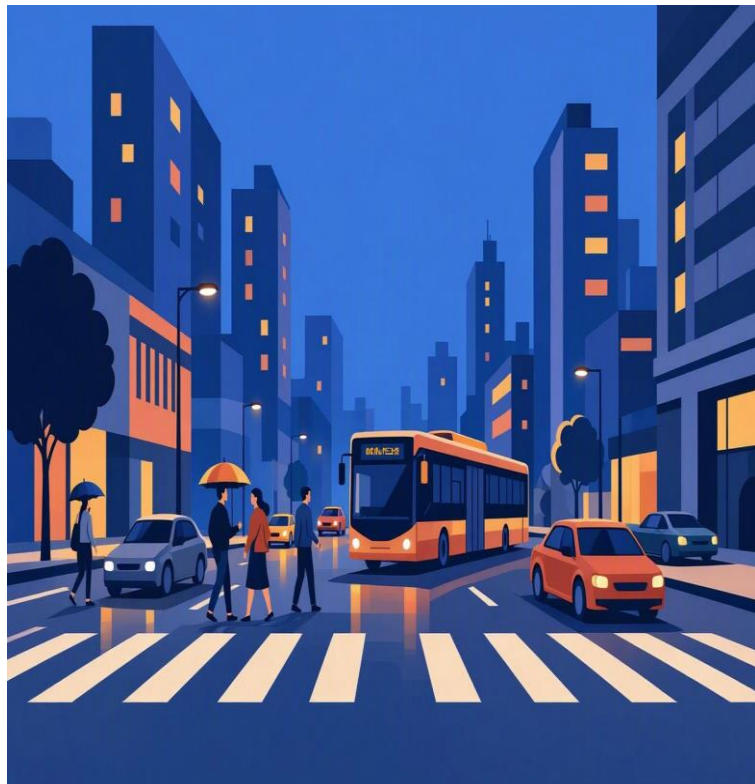
Moderator name

Netanel Ben Hamo

Table of Contents

1.	The Problem & Goal	3–4
2.	System Capabilities	5
3.	System Architecture & Data Flow	6
4.	Tier Deep Dives: Client, Server, Database & AI	7–10
5.	System Users & Personas	11
6.	Role-Based Dashboards	12–15
7.	Technology Stack	16
8.	Work Methodology & Sprints	17
9.	Existing Solutions vs. SafeCross	18
10.	Challenges & Engineering Solutions	19–20
11.	Conclusions & System Impact	21
12.	Further Development	22

The Problem: Road Safety Risks



High-Risk Zones

Pedestrian crossings and intersections account for a disproportionate share of urban traffic accidents and near-misses.

Ineffective Manual Monitoring

Human observers are unable to maintain consistent, continuous surveillance across multiple locations simultaneously.

Delayed Detection

There is an urgent need for immediate detection of pedestrians, especially children, in life-threatening situations.

GOAL



SafeCross

A next-generation real-time monitoring platform powered by video analysis and artificial intelligence — purpose-built to protect pedestrians at urban intersections.

Real-Time Video Analysis



Continuous monitoring of live camera feeds using AI-driven processing.

Automatic Hazard Detection



Identifies pedestrians, children, vehicles, and road hazards in milliseconds.

Smart Instant Alerts

Operators receive prioritized, actionable notifications the moment danger is detected.

System Capabilities



Interactive Map

Geographic deployment of all monitored intersections with live status indicators — giving operators a complete operational picture at a glance.



Smart Alert Table

Filterable by severity — Critical, Warning, or Normal — with full status management and escalation tracking.



Analytics Reports

Complete event history and trend analysis to support data-driven safety improvements over time.

System Architecture & Data Flow Overview

Client Tier (React & Vite)

- Responsive SPA running in the browser
- Role-Based Dashboards: Admin, Dispatcher, Manager, Technician.

Server Tier (Node.js & Express)

- RESTful API with structured MVC architecture & JWT security
- Socket.io: Real-time event broadcasting (Zero Page Refresh).

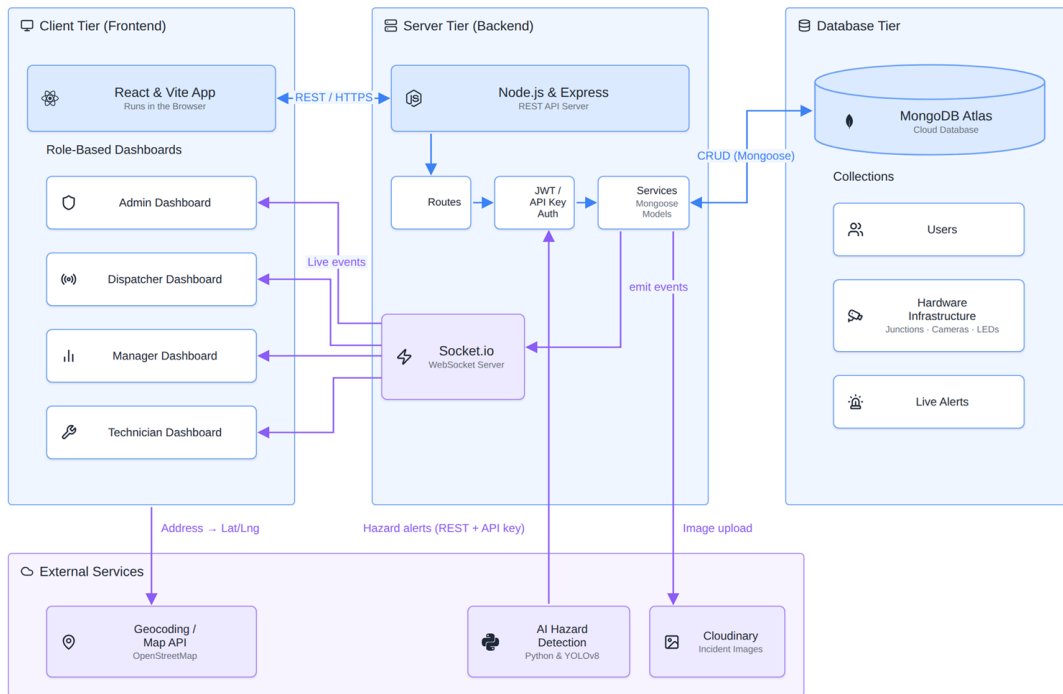
Database Tier (MongoDB Atlas)

- Cloud NoSQL document database managed via Mongoose ODM
- Stores Users, Hardware Infrastructure (Junctions, Cameras, LEDs), and Live Alerts.

External Services & AI

- Python & YOLOv8: Real-time AI video processing & hazard detection
- Geocoding API: Automated address-to-coordinate mapping for the maps interface.

SafeCross — Real-Time Smart Traffic Management System Architecture



Client Tier Architecture

React & Vite - Deep Dive

Component-Based SPA (React & Vite)

Modular development based on reactive components, managed under the high-performance Vite build tool, utilizing React Router for seamless page navigation.



```
<Router>
  <Routes>
    <Route path="/" element={<Login />} />
    <Route path="/dispatcher" element={<DispatcherDashboard />} />
    <Route path="/admin" element={<AdminDashboard />} />
    <Route path="/manager" element={<ManagerDashboard />} />
    <Route path="/technician" element={<TechnicianDashboard />} />
    <Route path="/crosswalk/:id" element={<CrosswalkDetails />} />
  </Routes>
</Router>
```

Role-Based Access & UI (RBAC)

Mechanism that isolates screen displays and user permissions according to their system role (Admin, Dispatcher, Manager, Technician).



```
if (role === 'admin') navigate('/admin');
else if (role === 'manager') navigate('/manager');
else if (role === 'technician') navigate('/technician');
else navigate('/dispatcher');
```

Dynamic State Management & Live Binding

Utilizing React Hooks (useState, useEffect) combined with Socket.io event listeners to automatically update component state and UI in real-time (Zero Page Refresh).



```
const socketUrl = apiUrl.replace('/api', '');
const socket = io(socketUrl);

socket.on('newAlert', (newAlertData) => {
  setAlerts((prevAlerts) => [newAlertData, ...prevAlerts]);
});
```

Responsive Design (Tailwind CSS)

A clean, modern, and responsive UI fully tailored both for data-heavy desktop dispatch centers and mobile field devices for technicians.



```
<div className="flex flex-col md:flex-row h-screen bg-slate-100 font-sans">
  <aside className="w-full md:w-64 bg-slate-800 text-white p-4 md:p-6 flex">
    <div>
```

Server Tier Architecture

Node.js & Express - Deep Dive

RESTful API & MVC Architecture

Structured backend using Express, separating routes, services, and database models for clean, maintainable code.



```
// routes → service → model
router.post('/', authenticate, authorize('Admin'), async (req, res) => {
  const saved = await createCrosswalk(req.body); // service layer
  res.status(201).json(saved);
});
export const createCrosswalk = async (crosswalkData) =>
  new Crosswalk(crosswalkData).save(); // Mongoose model
```

Security & Authentication (JWT)

Robust middleware enforcing secure route access using JSON Web Tokens (Authorization: Bearer <token>).



```
const authHeader = req.headers.authorization; // "Bearer <token>"
const token = authHeader.split(" ")[1];
const decoded = jwt.verify(token, process.env.JWT_SECRET);
req.user = decoded; // { userId, role }
next();
```

WebSocket Server (Socket.io)

An independent real-time communication layer executing bi-directional event broadcasting (newAlert, alertUpdated) across all connected clients.



```
const changeStream = Alert.watch([], { fullDocument: 'updateLookup' });
changeStream.on('change', (change) => {
  if (change.operationType === 'insert')
    getIO().emit('newAlert', change.fullDocument); // all clients
  else if (change.operationType === 'update')
    getIO().emit('alertUpdated', change.fullDocument);
});
```

Environment & Integration

Configuration management via dotenv and seamless connectivity with MongoDB Atlas and external AI service and Cloudinary.



```
import 'dotenv/config';
await mongoose.connect(process.env.MONGO_URI); // MongoDB Atlas
const AI_SERVICE_URL = process.env.AI_SERVICE_URL; // Python AI service
cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME, // Cloudinary
});
```

Database Tier & Data Modeling

MongoDB Atlas - Deep Dive

MongoDB Atlas (Cloud NoSQL)

High-availability, secure cloud database offering scalable document storage and real-time query performance.



```
const conn = await mongoose.connect(process.env.MONGO_URI); // Atlas cluster
console.log('MongoDB Connected: ${conn.connection.host}');

// Atlas replica set → real-time Change Streams
const changeStream = Alert.watch([], { fullDocument: 'updateLookup' });
```

Mongoose ODM

Elegant schema-based modeling for Node.js, enforcing data integrity, validation, and relation management.



```
const cameraSchema = new mongoose.Schema({
  name: { type: String, required: true },
  status: { type: String, enum: INFRA_STATUSES, default: 'active' },
  ip: { type: String, default: '' },
  junctionId: { type: mongoose.Schema.Types.ObjectId,
    ref: 'Crosswalk', required: true },
});
const Camera = mongoose.model("Camera", cameraSchema);
```

Structured Collections Architecture

Dedicated schemas for core system entities:

- **Users:** Role-based access control (Admin, Dispatcher, Manager, Technician) with hashed (bcrypt) credentials.
- **Hardware Infrastructure:** Spatial mapping data for junctions (coordinates, city, street), cameras, and LED indicators.
- **Live Alerts:** Historical and active emergency events to specific cameras and crosswalk.



```
// users
passwordHash: { type: String, required: true }, // bcrypt
role: { type: String, enum: ['Admin', 'Manager', 'Dispatcher', 'Technician'] },
// crosswalks (cameras & leds → junctionId)
city: { type: String }, lat: { type: Number }, lng: { type: Number },
// alerts
crosswalkId: { type: String, required: true },
severity: { type: String, enum: ['Low', 'Medium', 'High'] },
isResolved: { type: Boolean, default: false },
```

Database Tier & Data Modeling

External AI & Spatial Service Integrations

AI Hazard Detection (Python & YOLOv8)

- Real-time video stream processing to identify pedestrians, vehicles, and potential dangers at crosswalks.
- Computes confidence scores and risk levels (High/Medium/Low) and sends automated alerts to the server via secured REST API.



```
# YOLOv8: detect & track pedestrians / vehicles on each video frame
results = self.model.track(frame, persist=True, conf=config.CONFIDENCE_THRESHOLD,
                           classes=sorted(config.TARGET_CLASS_IDS))[0]

# Risk level (High / Medium / Low) + confidence score
level, name, reason = CASES[case_id]
confidence = int(round(100 * k.mean_confidence * min(1.0, k.n_obs / 3.0)))

# Send the alert to the backend via the secured REST API
self._session.headers["x-api-key"] = config.SENSOR_API_KEY
self._session.post(self.url, json=build_payload(event))
```

Geocoding & Spatial Services

- Converts textual address inputs (street and city names) into precise geographical coordinates (Lat/Lng).
- Enables seamless integration with interactive mapping interfaces (React-Leaflet) for the operations center.



```
let apiUrl = `https://nominatim.openstreetmap.org/search?format=json&country=israel`;
if (city) apiUrl += `&city=${encodeURIComponent(city)}`;
if (street) apiUrl += `&street=${encodeURIComponent(street)}`;

const response = await fetch(apiUrl);
const data = await response.json();

if (data && data.length > 0) {
  setInfraFormData({
    ...infraFormData,
    lat: data[0].lat.substring(0, 8),
    lng: data[0].lon.substring(0, 8)
  });
}
```

System Users & Personas

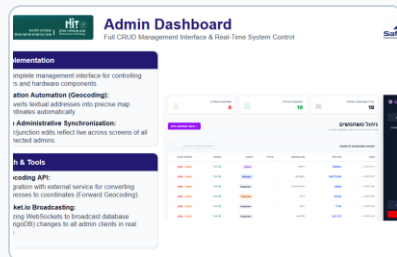
The system is divided into 4 distinct roles (Personas) based on permissions and operational responsibilities:

Admin

Role: Primary system administrator.

Capabilities:

Full management of system users (adding, editing, suspending, or deleting) and hardware infrastructure management—including adding and editing junctions, cameras, LED lights, etc.

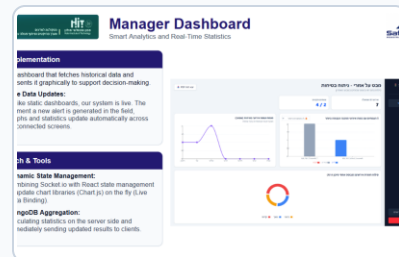


Manager

Role: Management tracking and analytics.

Capabilities:

Viewing statistics, analyzing data, and reviewing reports on incidents and events across various intersections over time to support data-driven decision making.



Dispatcher

Role: Real-time operational monitoring.

Capabilities:

Monitoring the interactive map, tracking the live alert table (updated in real-time via Socket.io), and handling critical anomaly events captured by the cameras.



Technician

Role: Field maintenance and troubleshooting.

Capabilities:

Receiving a list of daily maintenance tasks, navigating to intersections requiring attention, and updating task statuses and subtasks until issues are fully resolved.



Admin Dashboard

Full CRUD Management Interface & Real-Time System Control

Implementation

A full management interface (CRUD) for controlling users and hardware components, featuring two advanced capabilities:

Location Automation (Geocoding):

The system automatically converts textual addresses entered by the admin into precise map coordinates.

Live Administrative Synchronization:

Any addition or modification of a user or junction is reflected in real time across the screens of all other connected admins.

Tech & Tools

Geocoding API:

Client-side integration using React fetch to convert addresses into coordinates, offloading processing from the backend.

Socket.io Broadcasting:

Utilizing WebSockets to broadcast database (MongoDB) changes to all admin clients in real time.

משתמשים מושגים
0

משתמשים פעילים
10

סה"כ משתמשים רשומים
10

ניהול משתמשים

צפייה, יצירה ועריכת הרשאות משתמשי מערכת

חיפוש משתמש, ת.ז. או מזהה...

מזהה	שם מלא	שם משתמש	אימייל	תפקיד	סטטוס	פעולות אדמין
...6ab54ca	Admin	admin	-	Admin	פעיל	השגה מחק
...6abb7887	מנהל אזור	manager	-	Manager	פעיל	השגה מחק
...6abb7888	מוקדן	dispatcher	-	Dispatcher	פעיל	השגה מחק
...6abb7888	טכנאי	tech	-	Technician	פעיל	השגה מחק
...6abb919b	אדיר	adir	-	Dispatcher	פעיל	השגה מחק
...6abb9c88	דוד כהן	david1	-	Dispatcher	פעיל	השגה מחק

SafeCross Core

ניהול משתמשים

ניהול תשתיות חומרה

מחובר כעת

התנתק

Manager Dashboard

Smart Analytics and Real-Time Statistics

Implementation

A dashboard that fetches historical data and presents it graphically to support decision-making.

Live Data Updates:

Unlike static dashboards, our system is live. The moment a dispatcher marks an alert as handled, graphs and statistics update automatically and globally across the screens of all connected managers, with zero page refresh required.

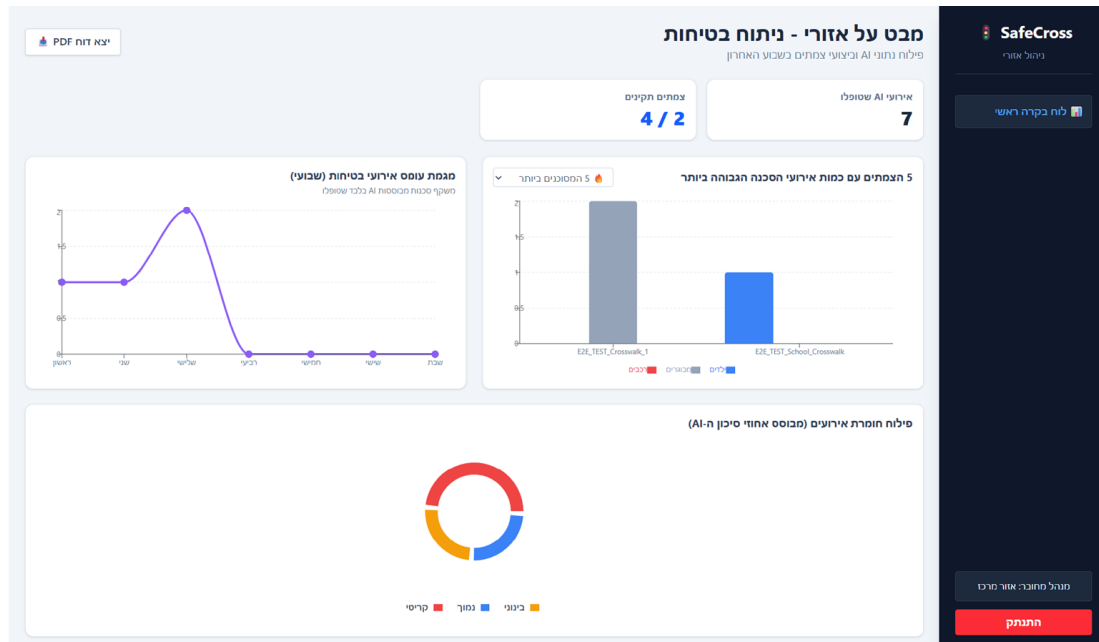
Tech & Tools

Dynamic State Management:

Combining Socket.io with React state management to update chart libraries (e.g., Chart.js) on the fly (Live Data Binding).

MongoDB Aggregation:

Calculating statistics on the server side. a Socket.io event tells clients to fetch the updated results.



Dispatcher Dashboard

Collaborative Operations Center and Live Monitoring

Implementation

A control center featuring an interactive map and an alert board fed directly from the cameras.

Collaborative Workspace:

If one dispatcher takes responsibility for an event and marks an alert as "handled," the status is immediately updated on the screens of all other dispatchers on shift. This prevents duplicated efforts and optimizes response times.

Tech & Tools

Real-Time Event Broadcasting:

Advanced use of Socket.io to broadcast a single user's actions to all connected clients (global event broadcast).

Mapping & Location:

Map integration utilizing the lat/lng coordinates stored for each junction.

The dashboard interface includes a top header with the SafeCross logo and a title 'מוקד שליטה בזמן אמת' (Real-time Control Center). Below the header is a map of Tel Aviv showing a highlighted area. To the left of the map is a sidebar with a list of events, including 'הרצל 45, חולון' and 'אבן גבירול 10, תל אביב'. To the right of the map is a table of events with columns for 'תאריך ושעה' (Date and Time), 'חומרה' (Severity), 'תיאור אירוע' (Event Description), 'ניתוח AI (פילוח)' (AI Analysis (Filtering)), 'מיקום (מאקרו ומיקרו)' (Location (Macro and Micro)), 'ציד מנסר' (Mantis Hunting), 'סטטוס סיפול' (Completion Status), and 'תיעוד' (Recording). The table shows two events: one at 24.9.2026 19:43 with a severity of 'בינוני' (Medium) and an AI analysis of '69% ביטחון' (69% Confidence), and another at 24.9.2026 19:43 with a severity of 'נמוך' (Low) and an AI analysis of '91% ביטחון' (91% Confidence).

Technician Dashboard

Field Maintenance and Operations

Implementation

A mobile-optimized interface for field workers. A technician receives a list of tasks, navigates to the locations, and closes tickets. The moment a technician updates a status in the field, the dispatcher and manager immediately see the change on their screens (Single Source of Truth)

Tech & Tools

Responsive UI: Mobile-First development using React, custom-tailored for small screens in the field.

Live Status Updates:

API calls that modify the issue's status, combined with real-time Push updates to the control center, ensuring full transparency with the field worker.

The screenshot displays the Technician Dashboard interface. At the top right, it says "ניהול תקלות חומרה" (Severe Issue Management) and "הצגת יצור קשר והודעות בזמן אמת" (Real-time contact and notifications). Below this is a map showing a city area with a red line indicating a route. On the left, there's a sidebar with "קריאת מידע" (View Information) and "התחבר" (Log In). The main content area shows a table of tasks with columns for "סוג רכיב / מיקום" (Component Type / Location), "צומת כנענות 70, חיפה" (Kana'an Junction 70, Haifa), "מצלמה (IP: 127.0.0.10)" (Camera), and "תאורת LED (red)" (LED Lighting). The table lists three tasks with their respective IDs and statuses.

סוג רכיב / מיקום	צומת כנענות 70, חיפה	מצלמה (IP: 127.0.0.10)	תאורת LED (red)
מזהה פערות <td>השאר התקלה <td>פערות סכאז</td> <td>מזהה פערות</td> </td>	השאר התקלה <td>פערות סכאז</td> <td>מזהה פערות</td>	פערות סכאז	מזהה פערות
6ab0c08efc92bec3c2e94a7	תקלה פערות סכאז	מזהה פערות	מזהה פערות
6ab088e7d1178068ab02c81	תקלה פערות סכאז (A)	מזהה פערות	מזהה פערות
6ab088e7d1178068ab02c81	תקלה פערות	מזהה פערות	מזהה פערות

Technology Stack

Overview of the tools, frameworks, and technologies utilized in the SafeCross system

Frontend

React (Vite): Modern component-based UI framework for building fast and reactive interfaces.

Tailwind CSS: Utility-first CSS framework for responsive and modern styling.

React Router: Declarative routing for seamless multi-page navigation.

Axios: Promise-based HTTP client for managing API requests.

Backend

Node.js & Express.js : Fast, lightweight server-side runtime and framework for building RESTful APIs

Socket.io: Enables real-time, bi-directional event-driven communication for live synchronization.

JWT (JSON Web Tokens): Secure authentication and authorization mechanism for route protection.

Database & Storage

MongoDB & Mongoose: NoSQL database with flexible document schemas and powerful object data modeling.

Cloudinary: Cloud-based media management platform for scalable storage of incident images.

Artificial Intelligence & Services

Python & YOLOv8: real-time hazard identification (children, pedestrians, vehicles). Deep learning object detection model for real-time hazard identification (children, pedestrians, vehicles).

Geocoding API: External service integration for automated address-to-coordinate mapping..

Work Methodology & Development Sprints

Sprint —1 Foundations & Environment

Team coordination via GitHub & VSCode.

Backend: Node.js, Express, MongoDB

Atlas. Frontend: React with Vite, Tailwind

CSS.

1

2

Sprint —2 API, Database & Security

Mongoose schemas (Users, Alerts,

Crosswalks, Cameras, LEDs), JWT

Authentication with custom middleware,

REST API integration via Axios.

3

Sprint —3 AI Integration & UI

YOLOv8 (Python) for real-time hazard

detection. Interactive role-based

dashboards for Admin, Manager,

Dispatcher, and Technician.

4

Sprint —4 Real-Time Sync & Cloud

Socket.io for live collaborative data sync.

Cloudinary for scalable incident image storage.

End-to-end testing and UI optimizations.

Existing Solutions vs. The SafeCross Approach

Traditional CCTV Cameras

Current Limitations:

Passive monitoring used mainly for post-accident investigations.

The SafeCross Advantage:

AI-driven behavioral analysis. Upon detecting dangerous pedestrian behavior, the system instantly flags the alert to activate a digital warning sign for approaching drivers, and generates analytics reports to help municipalities identify and address high-risk intersections.

Passive Pedestrian Crossings (Unsignalized)

Current Limitations:

Rely entirely on driver alertness. Drivers often fail to notice pedestrians waiting to cross, while pedestrians may step into the road unexpectedly.

The SafeCross Advantage:

Transforms passive crosswalks into active warning systems. The smart illuminated sign is designed not to trigger for every standard crossing; it is flagged for activation only when the AI detects unpredictable or dangerous behavior, demanding immediate driver attention exactly when it matters most.

Basic Motion Sensors

Current Limitations:

High false-alarm rates triggered by any general movement.

The SafeCross Advantage:

Cameras focus strictly on the pedestrian waiting area before the crosswalk. The AI classifies objects and behaviors specifically based on the predefined danger potentials configured in our system (e.g., distinguishing a child's unpredictable behavior significantly reducing irrelevant alerts).

Unexpected Challenges & Engineering Solutions

Evolving Project Requirements

The Challenge: Understanding the full scope of work while actively developing the application, leading to shifting requirements.

The Solution: Maintained agile workflows, held regular alignment meetings, and updated the development plan continuously.

Unforeseen Scenarios & Code Refactoring

The Challenge: During development, new events and edge cases constantly emerged that our initial architecture did not cover, requiring us to rewrite existing code.

The Solution: Established strict communication channels within the team, ensured full synchronization, and tackled complex changes collaboratively.

Database Schema Expansion

The Challenge: Expanding the database architecture (MongoDB/Mongoose) to accommodate new features without breaking existing data.

The Solution: Analyzed changing requirements carefully, mapped out schema updates before implementation, and performed rigorous testing.

Unexpected Challenges & Engineering Solutions

Complex Map & UI Integration

The Challenge: Collaborating on the interactive map and dashboard pages simultaneously without causing code conflicts.

The Solution: Utilized shared repositories, implemented code reviews, and focused on seamless integration between components

Multi-Role Real-Time Synchronization

The Challenge: Enabling live, two-way communication across 4 distinct user roles while transferring data instantly to sync all dashboards.

The Solution: Paused to conduct an in-depth study of Socket.io, accurately mapping out the data flow and role requirements before approaching the code itself.

Scope Management & Page Complexity

The Challenge: Building at least one dedicated, feature-heavy page per user role created a massive workload and a fear of missing critical system definitions.

The Solution: Adopted a highly focused iterative approach. We built the base infrastructure and finalized one page at a time before implementing the complex communication between them.

Conclusions & System Impact

Core Conclusions & Achievements:

End-to-End Real-Time System

Successfully developed and tested a multi-role monitoring platform combining React, Node.js, Python/YOLOv8, and MongoDB Atlas.

Transition from Passive Monitoring to Active Smart Warning

Upgraded the traditional approach of pedestrian crossings into an active system automatically triggered by AI only upon detecting real danger.

Human-Verified Alerts in Management Statistics

Integrating the dispatcher as a supervisory layer who reviews, verifies, and filters field alerts before they become official data, metrics, and graphs on the manager's dashboard - ensuring data-driven decision-making based on reliable data.

Operational & Collaborative Efficiency

Minimized response times and eliminated duplicated efforts through real-time synchronization (Socket.io) across Admin, Dispatcher, Manager, and Technician roles.

Conclusions for Further Development

AI Agent for Operator Action Learning

Introducing a learning mechanism where the AI agent learns which alerts were marked by dispatchers as handled or false positives, thereby continuously improving filtering accuracy and reducing false alarms.

Audit Logging & Analytical Reporting

Developing an advanced mechanism for recording event history and generating comprehensive reports to identify trends, pinpoint failure-prone junctions, and improve infrastructure over time.

Smart City Integration

Expanding deployment to additional municipalities and integrating directly with municipal traffic light controllers.

Dedicated Mobile App

Developing a native mobile application tailored specifically for field technicians to manage maintenance tasks on the go.

Thank you!

GitHub: https://github.com/yoska2/AI_Smart_Crosswalk.git

YouTube: